# 往年题精选：独立解析

历年题精选 · 独立解析 · 请先完成题目再核对

对应题目册，共44题。请先独立作答。以下解析由助手重新推导；原资料的错误、歧义与架构差异在相关条目中说明。未把原卷评分搬到重新编排的题库，也未收录2025第一次阶段测验的任何题目或答案。

## A 整数、位操作与字节序

### Y–A01 任意位宽的补码与大端存储

位宽 $w = 32N$，最小负数是 $-2^{w-1}$，位模式只有最高位为1。 大端：最低地址为 `0x80`，其余字节全为0。小端：最低地址为 `0x00`，最高地址为 `0x80`。 大小端改变字节的排列次序，不改变一个字节内部的位次序。

### Y–A02 字符串字节、截断与符号扩展

读取结果 `0x38313032`。右移得到 `0x00038313`，截断后为 `0x8313`，其补码值为 $33555 - 65536 = -31981$。 转换为32位 unsigned 得到 $2^{32} - 31981$，即 `0xFFFF8313`。从负的 short 到 unsigned 不能直接理解成高位全补0。

### Y–A03 异或交换逐行追踪

第一行：`x=a, y=a^b`；第二行：`x=a^(a^b)=b, y=a^b`；第三行：`x=b, y=b^(a^b)=a`。 因此交换成功。原文件先写了错误选项C，随后已修订为B。本题不采用未修订答案。

### Y–A04 从大端字节到小端整数的掩码

`ip` 的位模式为 `0xTT38A8C0`。掩码为 `0x00FFFFFF`，比较值为 `0x0038A8C0`。 大端排列的字节被小端整数读取，最先存放的C0变成最低有效字节。括号必须保留：C中 `==` 的优先级高于按位 `&`。

### Y–A05 连续整数的异或

每组 $(4k) \oplus (4k+1) \oplus (4k+2) \oplus (4k+3) = 0$。0到31共8组，最终为0。组内前一对异或为1，后一对也为1，再异或为0。

### Y–A06 负奇数右移与除法

$x = 8225, y = -8225$。`x/2=4112`， `x>>1=4112`； `y/2=-4112`， `y>>1=-4113`，所以 `z=1`。 除法向零截断；算术右移对应向负无穷取整。差异出现在负数且不能整除的情况。

### Y–A07 比较前先确定转换后的类型

依次为：真（unsigned）、假（unsigned）、假（unsigned）、真（int）。 第二式右侧转换为 `2147483648U`；第三式比较 `4294967295U < 4294967294U`。第四式已显式转回int，比较 `INT_MAX > INT_MIN`。 原题只有D的“真假及类型说明”都正确。补码转换模型是题设，不能推广为所有C实现的保证。

### Y–A08 lowbit命题遗漏了什么

依次为4、0、`0x80000000u`。x非零时结果保留最低的一个1，所以为某个 $2^k$；x=0时为0，不能写成 $2^k$。 使用unsigned也避开对有符号INT_MIN取负的未定义行为。原卷一2D的“任意x”漏掉了0，不能照着其选项答案背。

## B 浮点编码、舍入与范围

### Y-B01 没有符号位的小浮点格式

| 输入 | 编码 | 实际值 |
|---|---|---|
| 3 | `100 10` | 3 |
| 9 | `110 00` | 8 |
| 3/16 | `000 11` | 3/16 |
| 15/2 | `110 00` | 8 |

3为 $1.10_2 \times 2^1$。9在8和10中间，8的最低保留位为0。非规格化步长为 $2^{1-3-2} = 1/16$。7.5在7和8中间，7的有效尾数为111，向偶数舍入产生进位，得到8；不能只改frac而忘记调整exp。

### Y-B02 最小正规格化数为什么不含n

最小正规格化数为 $2^{1-\mathrm{Bias}} = 2^{2-2^{k-1}}$；最小正非规格化数为 $2^{1-\mathrm{Bias}-n}$。 前者有效尾数是1，后者为 $2^{-n}$；二者使用相同指数 $1-\mathrm{Bias}$，比值为 $2^n$。

### Y-B03 最大间距和半个ULP

间距是 $2^{127-23} = 2^{104}$，最大误差为半个间距 $2^{103}$。中点选择哪边由奇偶决定，但两边的距离都是半个间距。这里明确限制输入范围，否则“任意实数但不考虑Inf”的说法不够严谨。

### Y-B04 标准单精度编码

$27.5 = 11011.1_2 = 1.10111_2 \times 2^4$。s=0；exp=131=`10000011`；frac=`10111`后补18个0。 合起来为 **`0x41DC0000`**。这是数的位模式，若题目另问小端内存字节，则为 `00 00 DC 41`。

### Y-B05 浮点数加一什么时候失效

输出 **16777216，即** $2^{24}$。此前的正整数及加一结果均可精确表示；到 $2^{24}$ 后，右侧间距为2，$2^{24}+1$ 恰在中点，舍入回 $2^{24}$。 因此 `(f+1)-f` 得到0，触发退出。这不是到 $2^{23}$ 就失败：23位frac之外还有隐藏的1。

### Y-B06 连续五个整数的舍入误差

此处间距为4。

| a | f的实际值 | 误差 (int)f-a |
|---|---|---|
| 33554442 | 33554440 | −2 |
| 33554443 | 33554444 | 1 |
| 33554444 | 33554444 | 0 |
| 33554445 | 33554444 | −1 |
| 33554446 | 33554448 | 2 |

42和46都是中点，但方向不同：以步长4计，33554440和33554448对应的最低保留位为偶，33554444为奇。因此输出 `-2, 1, 0, -1, 2`。

### Y-B07 总位数不变，有限实数数量会不会变

总位数32不变。全1阶码共有 $2 \times 2^n = 2^{n+1}$ 个位模式，全部不是有限实数；+0和-0还要去重一次。 所以不同有限实数的数量是 $2^{32} - 2^{n+1} - 1$。n从23降为21，需扣除的特殊位模式减少，因此**更多**。但一般精度更低，数量更多不等于刻度更密。

### Y-B08 给定新规则后重新推导FP8

甲最大有限编码 `0 11110 11`，值为 $1.75 \times 2^{15} = 57344$；最大非规格化编码 `0 00000 11`，值为 $0.75 \times 2^{-14} = 3/65536$。 乙最大有限编码 `0 1111 110`，值为 $1.75 \times 2^{8} = 448$。若沿用IEEE式规则，则最大exp只能为1110，最大值为 $1.875 \times 2^{7} = 240$。448/240约1.867，若按原题要求取整则为2倍。 int8范围[-128,127]都在乙的范围内，不溢出；但精度不足，会舍入。例如17在16和18之间，最近偶数舍入为16。该题只按给定格式推理，不需要了解GPU。

## C 汇编指令、寄存器与条件码

### Y-C01 用lea组合乘法

第一条得到5n，第二条得到10n的低32位，所以填10。`lea`只做地址形式的算术，不读取该地址中的内存。若将此写回有符号C表达式，数学结果不可表示时不应声称C保证回绕。

### Y-C02 哪些指令会改CF

`shlw $1,%ax` 改CF，CF等于移出最高位的原bit15。jmp不读写条件码；jc读取CF但不改；inc会改部分条件码，却保留CF。不要把“会用条件码”和“会写条件码”混为一谈。

### Y-C03 test会把寄存器清零吗

sub、xor、and都可以；test不会写回AX，只计算按位与的结果来设置条件码。`test AX,AX` 常用来检查AX是否为0；它与“把AX变成0”是两件事。

### Y-C04 写32位寄存器和符号扩展

1、2真；3、4假。这里完整64位立即数的movabsq形式目标是64位寄存器。movswq是把16位有符号值扩展到64位；s表示符号扩展，w和q分别表示源16位、目标64位。

### Y-C05 语法合法不等于程序合理

只有B合法。A的立即数不能作目的；C的普通mov不能同时两端都为内存；D的比例因子只能为1、2、4、8。 B会把rsp清零，语法合法，但破坏普通调用环境的栈指针，后续push/call/ret通常无法正常工作。不要因为一条指令“很危险”就判断其语法非法。

### Y-C06 32位加法同时考寄存器和标志

低32位运算 `0xFFFFFFFF+1` 得到0，写eax同时把rax高32位清零，所以完整rax为0。 CF=1（无符号进位），ZF=1，SF=0，OF=0（有符号看是-1+1=0，没有溢出）。

## D 条件、循环与跳转表

### Y-D01 相对跳转从哪里开始加

`f0`作为8位有符号位移为-16。基准为下一条指令地址 `0x400022`，所以目标为 `0x400012`。不是从je自身起始地址减16。

### Y-D02 跳转表先做了加一

表索引为i+1，允许的原值为-1到7。映射为：

| i | -1 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|---|
| 目标 | L9 | L5 | L6 | L7 | L2 | L7 | L8 | L2 | L5 |

3和6直接进入默认路径。源代码也可能显式写出这两个case，但行为恰好和default相同；编译器可以合并。无符号ja同时排除了归一化后过大的值与原始值小于-1的情况。

### Y-D03 从cmov还原三目运算

填 `x+y`、`y-x`。eax先保存x+y，edx保存y-x。`cmp esi,edi` 根据x-y设置标志；有符号x>=y时，用edx替换eax；否则保留原eax。

### Y-D04 不能用条件传送掩盖非法访存

两者语义相同，NULL时返回0。不能无条件提前读取*p，NULL时可能触发故障。 也不能依赖内存源cmov在条件为假时跳过访存：x86的该形式仍会读取源操作数，条件控制的是是否更新目标。安全实现可用分支，或先选出保证有效的地址再读取；本题不要求掌握额外优化技巧。

### Y-D05 把循环变量映射到栈槽

填 `(%rdx,%rax)`、`jb`、`2`。比较是无符号x与b，不能用jl。

| 轮次 | 进入时x,y,m | b | 右移后的y | 比较结果 | 结束时x,y,m |
|---|---|---|---|---|---|
| 1 | 10,0,4 | 4 | 0 | 10>=4 | 6,4,1 |
| 2 | 6,4,1 | 5 | 2 | 6>=5 | 1,3,0 |

按指令次序先更新y，再在满足条件时加m；m每轮右移2位。练习目的在变量映射与控制流，不要求证明平方根算法。

### Y-D06 完整switch：跳转表、落入、除法偏置

| i | 表项地址 | 跳转目标 |
|---|---|---|
| 0 | 0x400690 | 0x4004d1 |
| 1 | 0x400698 | 0x4004b8 |
| 2 | 0x4006a0 | 0x4004bb |
| 3 | 0x4006a8 | 0x4004c0 |
| 4 | 0x4006b0 | 0x4004d4 |
| 5 | 0x4006b8 | 0x4004c3 |

case1落入case2，case3落入case5，case0落入default；i=4直接到默认加法。 逻辑右移31位提取符号位，结果为0或1；负数先加1再算术右移1位，修正直接向下取整的差异。此片段等效于C的有符号除以2。 foo(13,1)=(13-10)*8=24；foo(-8,3)=(-8+5)/2=-1。

## E 过程调用与递归

### Y-E01 第七个参数前还有返回地址

S处存放返回地址。第k个参数地址为 $S+8(k-6)$，所以第7个在S+8，第8个在S+16。栈向低地址增长，但沿入口栈向高地址查看时，依次看到返回地址、第7参、第8参。

### Y-E02 栈帧、对齐与leave的准确含义

设B=S−8，push后rbp=B。旧rbp位于B，返回地址位于B+8；局部区为[B−48,B)。call前rsp=B−48=S−56，模16为0。 在题设rsp没有另行变化时，leave可用加48再pop替代。一般意义是 `movq %rbp,%rsp; popq %rbp`；仅知道rbp未变，不足以保证“add固定常数”总是等价。

### Y-E03 八参数调用的压栈顺序

a7在rsp+8，a8在rsp+16。交换push顺序会交换被调用者看到的第7、第8参。 调用者通过add16回收这16字节。System V约定不能擅自用r10、r11替代栈传参；双方自定义约定是另一个问题。使用栈参数不意味着必须建立rbp帧指针，函数可以通过rsp和已知偏移定位。

### Y-E04 call *rax和call *(rax)差一层

`call *%rax`使用rax的值作为目标； `call *(%rax)`先以rax为内存地址，读取其中的函数地址再调用；AT&T中的间接调用要有星号， `call (%rax)`不是本题的正确写法。间接调用同样会把下一条指令地址压栈。

### Y-E05 递归追踪与每层栈帧

入口序列为 `(7,6)` → `(6,3)` → `(5,10)` → `(4,5)` → `(3,16)` → `(2,8)` → `(1,4)` → `(0,2)`。最后由n=0返回2，逐层原样返回，结果为2。 最深时8层。相邻两层rsp相差48字节：call压8字节返回地址，push保存8字节rbp，再分配32字节局部区。 尾调用优化可能复用当前帧，因此不能把“源代码递归次数”无条件等同于机器上的栈帧数量。这里的8层与48字节结论依赖题目给定汇编。

## F 数组、结构体与联合

### Y-F01 同一结构体放到不同ABI下

| 环境 | a | b | c | d | e | f | 原sizeof | 最小sizeof |
|---|---|---|---|---|---|---|---|---|
| 甲 | 0 | 4 | 8 | 16 | 20 | 28 | 32 | 28 |
| 乙 | 0 | 8 | 16 | 24 | 32 | 40 | 48 | 32 |

甲可按c,e,b,d,f,a排列，总字段25字节，向4对齐为28。乙也可按c,e,b,d,f,a排列，总字段29字节，向8对齐为32。最小值不可能小于这些总字段大小向结构体对齐要求取整的下界，而给出的顺序达到下界。 不要沿用原带答案版文字中误写的个别填充字节数，应按偏移逐项计算。

### Y-F02 union没有被写过的字节

不能。只能写成 `0x0123????`，还需两个高地址字节的值。只有补充“它们均为0”才是0x01230000。 union成员共享起始地址，不代表只写较短成员会自动清零整个union。本题是给定字节的推理，不要求执行读取未初始化C对象。

### Y-F03 double和char解释共享字节

小端字节依次为 `00 00 00 00 00 00 F0 3F`，s.c[2]位于偏移6，是F0。按signed char为-16，按unsigned char为240。 大端字节为 `3F F0 00 00 00 00 00 00`，偏移6为00，所以为0。原题没有交代这些前提，其答案文件也修订为多项可接受；本题通过分情况消除歧义。

### Y-F04 结构体偏移直接出现在指令里

依次为16、4、52。A的c/i/d/array偏移为0/4/8/16；数组占40字节，A大小56。B的array/d/c/i偏移为0/40/48/52，B大小也为56。 源指令先用“数组首偏移+下标乘4”读int，再写到B.i所在偏移52。

### Y-F05 从地址步长反推二维数组

初始列偏移为4C=40，故C=10。相邻写地址的差是 $4[B-1]=40$，故B=11。地址从base+40写到base+400，共10次；下一次rax为base+440而退出。 A不能唯一确定，只须A>=10。原题四个选项中只有(10,11,10)合法，但脱离选项不能声称A必定等于10。

### Y-F06 逐层解读数组指针的sizeof

A是含2个指针的数组，sizeof A=16；*A是其中一个“指向3个int数组”的指针，大小8；**A是3个int的数组，大小12；***A是int，大小4。总和40。 sizeof的数组操作数不会像普通表达式那样退化成指针。这里只分析类型和sizeof，不实际解引用未初始化指针。

### Y-F07 结构体加多层声明混合题

s的成员偏移为0、4、10、12、16，总大小20。p是含8个指针的数组；每个指针指向含4个student指针的数组。因此**p是4个指针的数组，大小32。 第三式以字节计为2*8=16；第四式以p元素为单位计为4。总和20+32+16+4=72。

## G 内存安全与防护

### Y-G01 把缓冲区加大是否就安全

D。只增加容量仍可能遇到更长输入。B在长度检查、终止符空间和调用方式正确时，可防止相应越界写；A检测部分会破坏canary的栈覆盖；C使地址猜测更困难。 A/C不等于修复所有越界，也不保证阻止每种攻击；原题的“避免方法”应理解为防护层次，不是绝对安全承诺。

### Y-G02 canary应检查相等还是大小

不合适。canary是完整位模式校验，应检查是否相等，不是有符号小于等于。可以 `cmp` 后 `jne .Lfail`；也可xor两个值后 `jne .Lfail`，只有完全相同才正常继续。 使用jle会错误放过许多已经变小的损坏值。

### Y-G03 读懂真实栈保护代码

栈副本在rbp-8。两份值相同时xor结果为0，ZF=1，je走正常返回；不同时ZF=0，调用失败处理。 不能保证所有局部数据都没变：越界可能只覆盖其他局部变量而未触及canary，或损坏之后又恢复了相同canary。它检查的是这个哨兵值，而不是逐字节证明整个栈完整。

### Y-G04 变长栈帧不是越界防护

变长栈帧用于容纳运行时大小才确定的局部数据，本身不是越界防护。 不可执行栈阻止把栈上的数据当代码执行；返回链若跳到原来就可执行的代码片段，并不需要从栈上取指令，所以这项措施不能单独阻止它。后一问是连接本课第8讲ROP的补充追问。